

Cascade Graph Neural Networks for Few-Shot Learning on Point Clouds

Yangfan Li[✉], Cen Chen[✉], WeiQuan Yan, Zhongyao Cheng, Hui Li Tan, and Wenjie Zhang[✉]

Abstract—Point cloud data, a flexible 3D object representation, is critical for various applications such as autonomous driving, robotics and remote sensing. Despite the recent success of deep neural networks (DNNs) on supervised point cloud analysis tasks, they still rely on tedious manual annotation of point clouds and cannot make predictions for new classes. Unlike few-shot learning for 2D images with the advantages of large-scale datasets and high-quality deep pre-trained models like ResNet, for 3D few-shot learning, obtaining discriminative representations of unseen classes with high intra-class similarity and inter-class difference is very challenging. To address this issue, this work proposes a novel cascade graph neural network for few-shot learning on point clouds, termed as CGNN, in which two cascade GNNs are adopted to extract the intra-object topological information and learn the inter-object relations respectively. To further increase the discriminability of point cloud features, we first design a novel discriminative edge label to model the intra-class similarity and inter-class dissimilarity based on channel-wise feature variance and class consistency. Second, we propose a novel few-shot circle loss which classifies the nodes into two subsets, i.e., support to support pairs and support to query pairs, and optimizes the pair-wise similarity on two subsets independently. Extensive experiments on benchmark CAD and real LiDAR point cloud datasets have demonstrated that CGNN improves accuracy by 5.98% over the state-of-the-art GNN-based few-shot classification methods.

Index Terms—Few-shot learning, graph neural networks, point clouds.

I. INTRODUCTION

WITH the rapid development of 3D acquisition technologies, 3D sensors, such as LiDARs, 3D scanners and RGB-D cameras, are becoming increasingly available and

affordable [1]. Compared with two-dimensional (2D) data (e.g., images), 3D data can provide more information, such as rich geometry, shape and scale information, and is not easily affected by changes in light intensity and other objects' occlusion. The 3D data can aid a range of applications, including autonomous driving, robotics, remote sensing, and medical treatment [1].

As a flexible representation of 3D data, point cloud can represent arbitrarily shaped 3D objects and environmental data. Analyzing point clouds [2], [3] and mining valuable information carried by 3D data has been a popular research direction. Unlike traditional approaches which typically extract features manually designed by domain experts [4], deep learning based methods [1], [5], [6], [7], [8] are able to learn better feature representations from the raw data, and achieve significant improvement in performance in various point cloud processing tasks. PointNet++ [6] developed a hierarchical architecture to capture multi-scale features. Reference [9] proposes a general method which transforms disordered point cloud data to a regular voxel network before processing by deep neural networks.

Most deep learning based point cloud analyzing methods adopt the fully supervised learning scheme. However, these fully supervised methods are highly dependent on tedious and expensive manually labeled data for training, and are unable to make predictions for categories that do not exist in the training set. Enlightened by humans who can easily distinguish similar objects by looking at only a few samples of the same class, researchers have investigated few-shot learning [10], [11], [12], [13], [14], [15], [16] to leverage knowledge learned from past tasks and few labeled data from new tasks to efficiently solve new tasks [17]. As shown in Fig. 1, unlike traditional learning methods which usually require a lot of labeled data for the single task training [18], the few-shot learning settings define source and target classes without overlap in class. The samples in the source class are heavily labeled and represent past knowledge. Only a small number of samples are with labels in each target class, representing unseen new tasks. The goal of few-shot learning is to learn knowledge from the source class and then exploit the learned knowledge to recognize the unseen target class with a few samples.

However, the few-shot learning scheme for point cloud still remains under-explored with the following challenges. First, unlike the regular 2D images, the 3D point cloud data is unordered and irregular in the European space and contains complex geometric information [1]. Second, few-shot learning for 2D images usually takes advantage of huge datasets and

Manuscript received 31 December 2021; revised 21 August 2022 and 23 December 2022; accepted 6 January 2023. Date of publication 31 January 2023; date of current version 2 August 2023. This work was supported in part by the Shenzhen Science and Technology Program under Grant RCBS20200714114943014, and in part by the Shenzhen Science and Technology Program under Grant JCYJ20210324123802006. The Associate Editor for this article was W. Wei. (Corresponding author: Cen Chen.)

Yangfan Li is with the School of Computer Science and Engineering, Central South University, Changsha, Hunan 410083, China (e-mail: yangfanli@hnu.edu.cn).

Cen Chen is with the Infocomm for Research Institute, Singapore 138632, and also with the Shenzhen Research Institute, Hunan University, Shenzhen 518000, China (e-mail: chenc@i2r.a-star.edu.sg).

WeiQuan Yan is with the Peng Cheng Laboratory, Shenzhen 511464, China (e-mail: ywqqqqqq@outlook.com).

Zhongyao Cheng and Hui Li Tan are with the Infocomm for Research Institute, Singapore 138632 (e-mail: cheng_zhongyao@i2r.a-star.edu.sg; hltan@i2r.a-star.edu.sg).

Wenjie Zhang is with the Department of Electrical Engineering, The Hong Kong Polytechnic University, Hong Kong (e-mail: wenjie_zhang@u.nus.edu).

Digital Object Identifier 10.1109/TITS.2023.3237911

1558-0016 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

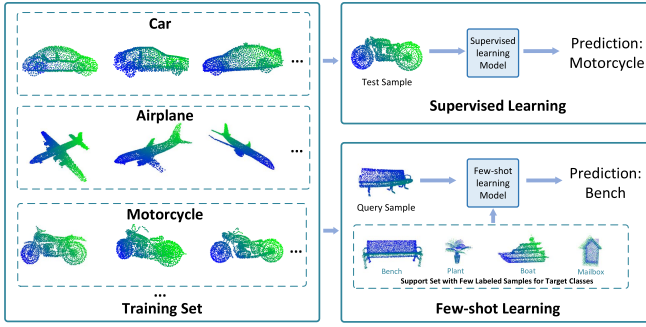


Fig. 1. Comparison of the traditional supervised learning and few-shot learning for 3D point cloud classification. Few-shot learning can recognize the query 3D sample whose class is not in the training set with a few labeled support samples.

high-quality deep pre-trained models like ResNet. In contrast, there is no parallel for 3D few-shot learning and the benchmark datasets are usually small in scale with limited sets of samples and classes [19]. For example, The number of classes and samples for the benchmark point cloud dataset such as ModelNet40 and ShapeNet is usually smaller than one-tenth of the numbers of the image dataset such as ImageNet. As a result, learning discriminative representations of unseen classes for 3D point cloud with high intra-class similarity and inter-class difference is more challenging compared with 2D images.

In this work, we propose CGNN, a novel cascade GNNs for few-shot point cloud classification. It mainly contains two graph neural networks which work in a cascade manner to model both the intra-object features and inter-object relations. The intra-object feature extraction GNN (IFE-GNN) is designed to extract geometric features from origin point clouds by constructing a directed k -nearest neighbor (KNN) graph. The inter-object relation reasoning GNN (IRR-GNN) exploits the GNNs to model the relation of objects. To efficiently extract the discriminative geometric features from small-scale point cloud datasets, we are motivated by [20] and [21] which suggests that different channels can carry different semantic information, and design a novel discriminative edge label to model the intra-class similarity and inter-class dissimilarity based on channel-wise feature variance and class consistency in the reasoning process. Based on the observation that the similarity scores of the support to support pair and support to query pair are quite different in the value distributions, we also propose a novel few-shot circle loss which classifies the nodes into two subsets, i.e., support to support pairs and support to query pairs, and optimize the pair-wise similarity on two subsets independently. As far as we know, CGNN is the first work fully utilizes GNNs for few-shot point cloud learning. We list the contributions as follows:

- We propose a novel fully GNN based methods, CGNN, for few-shot learning of point cloud classification. Two GNN network models work in a cascaded manner for both the intra-object topological feature extraction and the inter-object relation reasoning of the point cloud objects.

- We design a novel discriminative edge label to model the intra-class similarity and inter-class dissimilarity based on channel-wise feature variance and class consistency.
- We propose a novel few-shot circle loss that classifies the nodes into two subsets, i.e., support to support pairs and support to query pairs, and optimizes the pair-wise similarity on two subsets independently.
- Extensive experiments on two public few-shot point cloud datasets have demonstrated that CGNN achieves state-of-the-art performance for few-shot learning on point clouds. Extensive ablation experiments have also demonstrated the effectiveness of all the proposed schemes in CGNN.

The remaining of the work is organized as follows: In the next section, the related work is discussed. Section III presents the details of CGNN. Section IV presents the experimental results, while the work is concluded in section V.

II. RELATED WORK

This section discusses the related work for GNNs, few-shot learning, deep learning for point cloud as follows.

A. Graph Neural Networks

Recently, the graph neural network (GNNs) [22], [23], [24] have applied CNNs to irregular graph data, which has attracted widespread attention. The key idea is aggregating features from neighboring nodes in the graph, thereby can mine the relationships and interactions among graph nodes [14]. GNNs usually have 3 stages in the processing, i.e., edge update, aggregation, and node update. Variants of GNNs design different operations in these stages. Graph attention network (GAT) [23] leverages the attention mechanism to dynamically determine the importance of each neighbor in the neighborhood aggregation. GraphSAGE-Pool [25] contains an edge update stage to transform the edge features and utilizes max pooling as the aggregation operation. Graph convolutional network (GCN) [22] calculates the average results of the neighbor messages in its aggregation stage. Our CGNN is also designed based on the structure of GNN. It mainly contains two GNNs that work in a cascade manner to model both the intra-object features and inter-object relations. We explore the multi-granularity use of GNNs to process point cloud data. First, each point in a point cloud object can be a node in a graph, and GNN is designed for topology information extraction. Second, each object can also be a graph node and the GNN is leveraged for relation reasoning.

B. Few-Shot Learning

Researchers have investigated few-shot learning to leverage knowledge learned from past tasks and few-labeled data from new tasks to efficiently solve new tasks. The few-shot learning methods mainly fall into 3 categories. The metric-based methods [12], [13], [15] use the distance between support samples and query samples obtained during training for label inference of query samples. The matching network [15] proposes episode training to solve the few-shot learning problem, applying the feature similarities obtained during training

to the samples in the test environment. The prototype network [12] transforms all samples into a shared feature space and removes outliers based on the centroids of the support classes, and finally computes the common features of all samples in the same class for label prediction of new samples. The optimization-based methods [26], [27] first obtain an initialized model, and then learn the model parameters for the unseen target classes with few labeled samples. The relation-based methods [10], [13], [14], [28] aim at learning the relations between the objects of the query and support classes and making predictions based on the learned relations. CTM [28] proposed a feature selection scheme to remove the irrelevant dimension of features by traversing intra- and inter-class samples. The relation network [13] learns to measure the pair-wise relations of the query and support samples through the embedding and the relation modules.

Recently, GNNs have also shown promising results for modeling the relations in few-shot learning settings. These methods constructed a fully connected graph with the query items and the support items and utilize GNN to learn the pair-wise similarities and relations between the nodes. Reference [14] proposed a node label-based method for few-shot learning while EGNN [10] further introduces edge labels to improve the classification accuracy. However, they may fail to model the 3D point cloud data has richer feature relations such as topological information. HGNN [29] proposes a hierarchical GNN structure for few-shot learning. Our CGNN falls into this category and proposes a novel feature discriminative scheme in the EdgeUpdate and the NodeUpdate stage for relation reasoning.

C. Deep Learning for Point Cloud

Although deep learning has achieved great success on various domains [30], [31], [32] for regular data analysis, deep learning for irregular data such as graphs [33] or 3D point cloud [1] is still challenging. Point cloud is a commonly used format to represent 3D data without any discretization in the 3D space. As Deep learning technologies achieve impressive progress in 2D data, people pay attention to related research on point cloud data [1]. However, the structure of input data for standard deep learning models is always regular while point cloud data is irregular. Specifically, the same group of the points distributed in the 3D space have different permutations while they actually represent the same target. Recently, deep neural networks for point cloud data are designed to overcome this challenge [1], [5], [6], [7], [8]. Pointnet [5] and Pointnet++ [6] used MLP to model each point independently and subsequently adopted a symmetric aggregation function to calculate the global representation. Most existing work focused on supervised learning for 3D point clouds, very few works studied few-shot learning for 3D point clouds and it is still under-explored. Cheraghian et al. [19] identified the availability of high-quality pre-trained models, the hubness problem and the domain shift problem in 3D zero-shot learning and tried to address these issues. SFNet [34] proposed a self-supervised method to hierarchically encode the partitions of the point clouds with a cover tree. Deep learning for point cloud is crucial for service providing [35] in autonomous

driving and so on. Nevertheless, none of these methods explore the GNNs to learn the discriminative representations.

III. METHODOLOGY

A. Problem Definition: Few-Shot Point Clouds Classification

Recently, meta-learning based approach has been widely used to deal with few-shot learning problems. We follow the classic episodic training framework which has been widely utilized for few-shot learning [12], [13], [15], [26]. In the training stage, given a training set $\mathcal{C}_{train}$, in which each training example consists of a query set Q and a support set S . Both the query and support sets consist of multiple point cloud samples with labels. Each sample is denoted as x_i, y_i where x_i denotes the point cloud sample representing an object (pedestrian, car and etc.) and y_i denotes the class label which is known for the training. The objective is to train a classifier that can categorize the labels of the query set into the classes of the support set. If the support set contains N different categories and each category contains K examples, this few-shot classification task is defined as an N -way- K -shot classification task. In this testing stage, we have a test set in which the test example also contains a query set Q and a support set S . We use the same number of N and K with the training stage. Different from the training stage, the classes of the test set are unseen and different from the classes in the training set. The labels are known for S while unknown for Q . The trained classifier is leveraged to predict which class of the support sets the query set belongs to. The performance is evaluated based on prediction accuracy.

B. Framework Overview

CGNN is a cascade graph neural network for few-shot point cloud classification. Shown in Fig. 2, it mainly contains two graph neural networks which work in a cascade manner to model both the intra-object features and inter-object relations with few training examples.

1) Intra-Object Feature Extraction GNN (IFE-GNN):

Point clouds are flexible 3D representations. But the point clouds inherently lack topological information, we design the IFE-GNN built on DGCNN [36] to extract features from origin point clouds while recovering the topology to enrich the representation power of point clouds. In IFE-GNN, a directed k -nearest neighbor (KNN) graph is constructed from the point cloud of an object, and each node denotes a point in the point cloud. Then we perform EdgeConv operation for geometric feature extraction.

2) Inter-Object Relation Reasoning GNN (IRR-GNN):

Traditional deep learning based approaches for point cloud classification [5], [6] directly use the extracted features of the input point clouds to determine their categories. However, these methods require a large number of training examples and parameters to memorize the features of each category. Moreover, they fail to classify categories that are not in the training sets. In contrast, we exploit the graph neural networks to model the relation of objects of different categories. Each node represents a point cloud object and the edges denote their relations. A novel discriminative edge label is proposed to

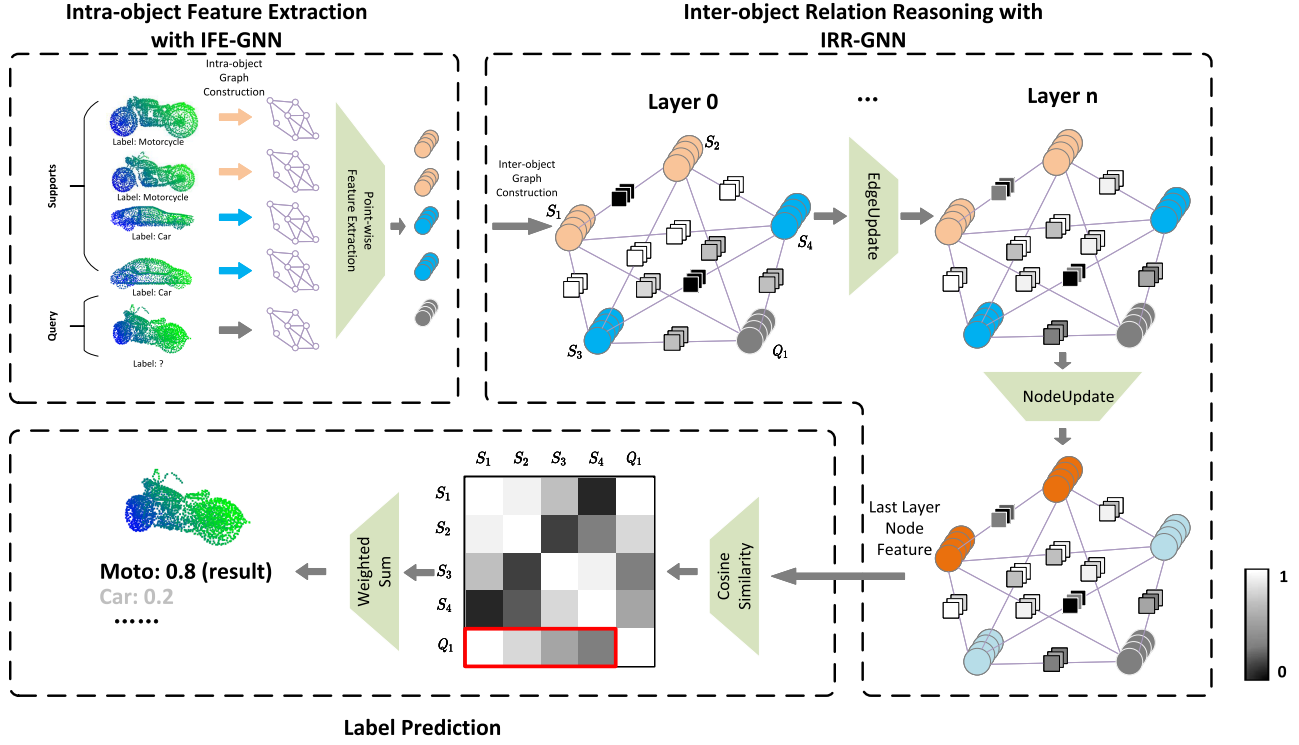


Fig. 2. Framework overview of CGNN with a 2-way 2-shot example (4 supports and 1 query). Nodes with the same color represent that they have the same class label. Nodes in grey denote the unlabeled query objects. The squares on the edge represent the edge labels and the shade of gray represents the (dis)similarity of each node at each channel.

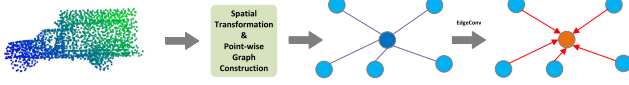


Fig. 3. Illustration of intra-object feature extraction. The point cloud data of an object is constructed as a graph and GNNs are utilized to capture the point-wise topological information.

describe the channel-wise similarity of the node relations. The label information is propagated from labeled samples to unlabeled query point clouds through the proposed EdgeUpdate and NodeUpdate operations.

We classify the point cloud objects through similarity-based relation reasoning of the input point clouds (query) and other objects (supports). The unlabeled queries can be classified into the class of labeled supports with the highest similarity. The network is end-to-end trainable with a novel few-shot circle loss which maximizes the intra-class feature similarity and the inter-class feature dissimilarity.

C. Intra-Object Feature Extraction GNN (IFE-GNN)

Point clouds are accurate 3D representation of objects. But the point clouds inherently lack topological information, we design the IFE-GNN built on DGCNN [36] to extract features from origin point clouds while recovering the topology to enrich the representation power of point clouds. Given an d -dimensional point cloud $X = \{x_1, x_2, \dots, x_n\}$ with n points to describe an object, where $x_i \in \mathbb{R}^d$ denotes each point in the point cloud. In the most basic setting of $d = 3$, each point

is represented with the 3D coordinates. Also, more features can be included such as color and surface normal to describe the points with a higher dimensional d . We build a local neighborhood graph $G_I = (V_I, E_I)$ to represent the point cloud, where $V_I = \{1, 2, \dots, n\}$ and $E_I \subseteq V_I \times V_I$ are the vertices and edges respectively. The graph construction is as follows. For each point v_i , we measure the euclidean distance of its features to other points and select K nearest points as the local neighbors in the local neighborhood graph. Specifically, the neighbors set $N_I(v_i)$ is selected as follows:

$$idx = \text{topK}(\text{distance}, K), \quad (1)$$

$$N_I(v_i) = \{v_j | j \in idx\}. \quad (2)$$

where idx denotes the index set with size K for the selected points. Note that, the node features are updated in each layer, and its feature distance from other points also changes after each layer. Therefore, the graph structure of each IFE-GNN layer is updated dynamically after each layer.

Then we connect each vertex $v_i \in V$ with its nearest neighbor $v_j \in N_I(v_i)$ to construct the graph G_I . Finally, we update the edges connecting the pairs of points by EdgeConv operation for the i -th vertex as follows:

$$x' = \sum_{j:(i,j) \in E_I} h_\theta(x_i, x_j), \quad (3)$$

where $h_\theta : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$ is nonlinear transformation with learnable parameters θ . In our IFE-GNN, we adopt convolution and max-pooling as the updating method.

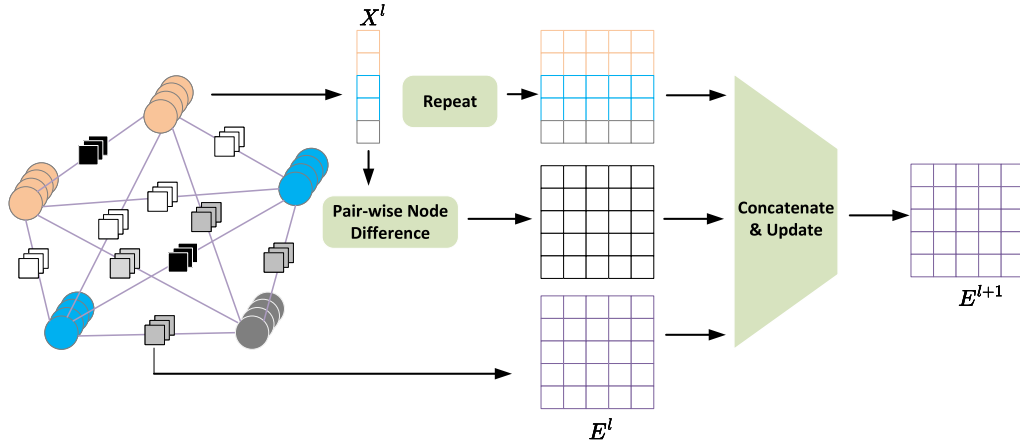


Fig. 4. Illustration of EdgeUpdate operation.

Algorithm 1 The Process of IRR-GNN for Inference

```

1: Input:  $G = (G, E)$ ,  $S = \{(x_i, y_i)\}_{i=1}^{N \times K}$ 
    $Q = \{x_i\}_{i=N \times K+1}^{N \times K+H}$ 
2: Parameters:  $\{\theta_e^l, \theta_n^l\}_{l=1}^L$ 
3: Output:  $\{\hat{y}_i\}_{i=N \times K+1}^{N \times K+H}$ 
4: Initialize:  $v_i^0 = f_{IFE}(x_i)$ 
5: for  $l = 1, \dots, L$  do
    $\triangleleft$  Edge Feature Update
6:   for  $(i, j) = 1, \dots, |E|$  do
7:      $e_{ij}^l \leftarrow \text{EdgeUpdate}(v_i^{l-1}, e_{ij}^{l-1}, \theta_e^l)$ 
8:   end for
    $\triangleleft$  Node Feature Update
9:   for  $i = 1, \dots, |V|$  do
10:     $v_i^l \leftarrow \text{NodeUpdate}(v_i^{l-1}, e_{ij}^l, \theta_n^l)$ 
11:   end for
    $\triangleleft$  Node Similarity Calculation
12:    $\text{sim}^l = \text{cosine\_similarity}(\{v_i\}_{i=1}^{|V|})$ 
    $\triangleleft$  Label Prediction
13:    $\{\hat{y}_i\}_{i=N \times K+1}^{N \times K+H} \leftarrow \text{predict}(\{\text{sim}^l\}_{j=0}^L)$ 
14: end for

```

D. Inter-Object Relation Reasoning GNN (IRR-GNN)

Each IRR-GNN layer contains an EdgeUpdate and NodeUpdate operation to update node features. The discriminative edge label describes the channel-wise similarity relation of a node pair with the channel-wise feature variance and class consistency, which facilitates the rich geometric information to propagate from labeled samples to unlabeled query point clouds. The labels of the query nodes are predicted based on the similarity matrix. The inference algorithm of IRR-GNN is summarized in Alg. 1.

1) *Analysis:* Most existing GNN algorithms such as GCN and GAT update the node features by aggregating the neighborhood node features without edge labels in their message passing process. In our few-shot point cloud classification problem, we classify objects through similarity-based relation reasoning and we expect the similarity information to be propagated to other nodes in the message passing. In this regard, the traditional message passing which relies only on the node label does not work well for this similarity-based reasoning, because it models the intra-class similarity and

inter-class dissimilarity in an implicit manner only with the node label. Moreover, compared with few-shot classification methods in the 2D domain, the 3D objects usually have complex geometric features which introduce more challenges for similarity-based reasoning.

References [20] and [21] suggest that different channels can carry different semantic information. Motivated by this, we think the channel-wise label is promising to model the complex geometric similarities point cloud and we aim to propagate the channel-wise intra-class similarity and inter-class dissimilarity in an explicit manner. The proposed discriminative edge label carries the similarity information in two aspects, i.e., the channel-wise feature variance and class consistency. The edge label with similar information is propagated through the proposed EdgeUpdate operation.

2) *Graph Construction and Initialization:* We first construct a graph for the input query and support point clouds in which each node in the graph represents a point cloud of an object and the edges denote their relations. Fig. 2 illustrates the graph construction procedure of a few-shot point cloud classification problem. In detail, the constructed fully-connected graph is defined as $G = (V, E, \tau)$, where τ denotes the N -way K -shot few-shot classification task, each node V_i denotes each point cloud sample x_i , and each edge $E_{(i,j)}$ denotes the relationship of node pair x_i and x_j . $|V| = N \times K + H$ is the total number of nodes. H is the number of query nodes. For each V_i , its feature is initialized as the output of the proposed IFE-GNN as:

$$v_i = f_{IFE}(x_i), \quad (4)$$

where f_{IFE} is the proposed IFE-GNN and v_i is a d -dimensional vector.

3) *EdgeUpdate Layer With Discriminative Edge Label:* We design the discriminative edge label to model the intra-class similarity and inter-class dissimilarity based on channel-wise feature variance and class consistency. We assign each edge with a d -dimensional edge label $e_{i,j}$, where each dimension represents the channel-wise similarity of the connected nodes v_i and v_j . All the dimensions of the first layer edge labels $e_{i,j}^0$ are initialized based on the class consistency. Specifically,

for the edge between two support nodes, the initial values are 1 if the nodes belong to the same class, otherwise they are initialized to 0. For the edge connecting a query node and other nodes, including support nodes and other query nodes, the edge labels are 0.5 because the query nodes are unlabeled. In this way, the class consistency information is encoded as edge labels for the following propagation. Formally,

$$e_{i,j}^0 = \begin{cases} 1, & \text{if } y_i = y_j \text{ and } i, j \leq N \times K \\ 0, & \text{if } y_i \neq y_j \text{ and } i, j \leq N \times K \\ 0.5, & \text{otherwise} \end{cases} \quad (5)$$

Then, we represent the channel-wise feature variance in a explicit manner as follow:

$$\mathbf{x}_{diff} = \mathbf{x}_i - \mathbf{x}_j, \quad (6)$$

Finally, we use the EdgeUpdate operation to update the edge labels in a channel-wise manner, and the process is shown in Fig.6(a). In each EdgeUpdate block, the feature variance $\mathbf{X}_{diff}^l \in \mathbb{R}^{d \times |V| \times |V|}$, the nodes feature $\mathbf{X}^l \in \mathbb{R}^{d \times |V|}$ and the edges labels $\mathbf{E}^l \in \mathbb{R}^{d \times |V| \times |V|}$ are concatenated to generate $\hat{\mathbf{E}}^l \in \mathbb{R}^{3 \times d \times |V| \times |V|}$, note that $\mathbf{X}^l$ is repeated $|V|$ times to match the dimension for concatenation. $\hat{\mathbf{E}}^l$ is then updated with convolution and Tanh activation. Note that the channels of the convolution kernel correspond with first dimension of $\hat{\mathbf{E}}^l$, and the number and size of the kernel are both 1, which indicates that we update the similarity score for the node pairs independently at each channel. 3 EdgeUpdate blocks are stacked and a diagnose mask is used to mask the edge features on the diagonal to generate the next layer discriminative edge label $\mathbf{E}^{l+1}$.

4) *NodeUpdate Layer*: The NodeUpdate layer is designed to aggregate the inter- and intra-class information from the connected nodes to obtain a new node representation. To achieve this, we propagate the information of the connected edges and aggregate the newly updated connected edge features which contain the channel-wise node (dis)similarity information as follows:

$$\mathbf{v}_{aggr} = \sum_{u \in N(v)} \mathbf{e}_{u,v}^l, \quad (7)$$

where $N(v)$ denotes the neighbor set of node v . The aggregation process is illustrated in Fig. 5. Then the old node features are updated through the NodeUpdate network shown in Fig. 6(b) with the aggregated feature $\mathbf{v}_{aggr}$. After aggregation, the aggregation feature $\mathbf{v}_{aggr}$ is concatenated with the old node features and processed with several layers of NodeUpdate block. We adopt the 1D convolution with here kernel size 1 here to update the node features in the NodeUpdate block. The LeakRelu is used as the activation function.

5) *Similarity-Based Label Prediction*: After L number of the EdgeUpdate and NodeUpdate layers. The class of the query objects can be obtained by measuring the feature similarity of the query object with the support objects. In CGNN, we calculate the node-wise cosine similarity after each NodeUpdate layer and use the average similarity of all the layers for prediction. Finally, the output label of the query objects

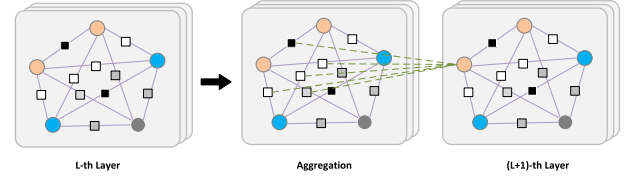


Fig. 5. Illustration of aggregation in the node update operation. The updated edge features are aggregated to update the node features.

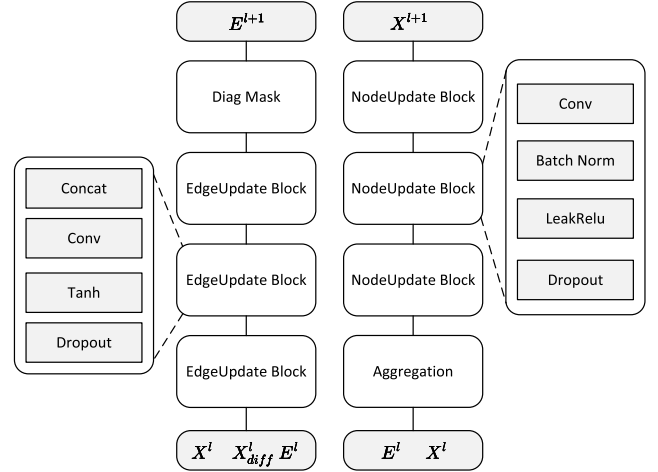


Fig. 6. Detailed network structure. (a) EdgeUpdate network. (b) NodeUpdate network.

corresponds with the class label of the support object with the highest similarity.

E. Few-Shot Circle Loss

Our CGNN can be trained in an end-to-end manner. The parameters in the feature extraction network (IFE-GNN) and the relation reasoning network (IRR-GNN) are co-optimized. Previous works such as EGNN often leverage independent binary cross-entropy (BCE) loss for each relationship of nodes. However, in the optimization objective of the BCE loss function, the intra-class similarity and inter-class dissimilarity use the same metric space, so it is not possible to reduce the intra-class similarity and increase the inter-class dissimilarity separately. Circle loss [37] is a promising optimization scheme that provides a clearer convergence object for pair-wise similarity optimization. Based on circle loss, we propose a novel few-shot circle (FSC) loss which considers the relationship for each node pair to supervise the relation learning. Given a set of node pairs P , the circle loss on P is defined as:

$$L_{cl}^P = \log[1 + \sum_{i=1}^M \sum_{j=1}^N \exp(\gamma (\alpha_n^j (s_n^j - \Delta_n) - \alpha_p^i (s_p^i - \Delta_p)))] \quad (8)$$

where M and N denote the number of pairs with the same/different labels in P respectively, $\{s_n^j\} (j = 1, 2, \dots, N)$ and $\{s_p^i\} (i = 1, 2, \dots, M)$ are the predicted pair-wise similarities and dissimilarities, α_n^j and α_p^i are the non-negative weighting factors corresponding the predictions, Δ_n and Δ_p

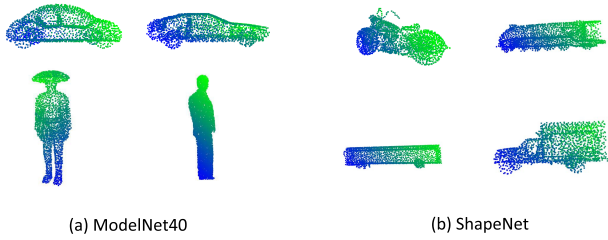


Fig. 7. Dataset visualization.

are the margins, and γ is the scaling factor. We do not directly apply circle loss on E because we observe that, in the settings of few-shot classification, the similarity scores of the support to support pair (s2s) and support to query pair (s2q) are quite different in the value distributions. The s2s scores are close to 0 and 1, while the s2q scores float around 0.5, mainly because the supports are labeled and the query is unlabeled. To avoid possible noise introduced by the distribution differences, we divide the edges E of the constructed graph in section III.D into two subsets, i.e., denoted as E^{s2s} and E^{s2q} and calculate the circle loss on the two sets respectively. The few-shot circle loss is defined as follows:

$$L = L_{cl}^{E^{s2s}} + L_{cl}^{E^{s2q}}. \quad (9)$$

IV. EXPERIMENTAL RESULTS

We have conducted extensive experiments to compare our proposed CGNN with some state-of-the-art methods for few-shot point cloud classification and ablation studies to demonstrate the effectiveness of all the proposed schemes in CGNN.

A. Experimental Setup

1) *Dataset Description*: We evaluate our CGNN on the commonly used datasets for point cloud classification, ModelNet40 [38] and ShapeNet [39] which contains, and a real-world LiDAR point cloud dataset in urban environment, i.e., Sydney Urban Objects. We visualize some point cloud objects in ModelNet40 and ShapeNet in Fig. 7.

a) *ModelNet40*¹: There are 12,308 pre-aligned shapes from 40 categories which include cars, pedestrians, airplanes and so on in the dataset. We sampled 1024 points uniformly from the surface of each object and scaled them to fit the unit sphere. We remove the original surface of the object and use the (x, y, z) coordinates of the sampled points to represent the object. We randomly chose 20 classes with 5,147 samples as training sets data, the other 20 classes, which contains 7,161 samples, are used as novel class for testing.

b) *ShapeNet*²: This is a common large scale 3D object classification benchmark. It contains with 51,127 objects in 55 categories which includes cars, pedestrians, trains, vessels and so on. 2048 points are used to describe each object. We randomly chose 27 classes with 19,882 samples as training sets data, the other 28 classes with 31,245 samples are used as novel classes for testing.

c) *Sydney Urban Objects*³: This point cloud dataset consists of 588 objects in 14 categories (vehicles, pedestrians, signs, trees and so on) scanned with a Velodyne HDL-64E LIDAR in the CBD of Sydney. Following [34], we pre-process this dataset. After pre-processing, 374 objects (each with 100 points) from 10 classes are reserved.

2) *Evaluation Metric*: The experiments of standard 5-way 5-shot few-shot classification tasks are conducted to evaluate the performance of our CGNN and other baselines. We used the same settings in EGNN [10] for fair comparisons. 5 queries are sampled for each of the 5 categories in the test episodes. For performance evaluation, we used 500 episodes which are generated randomly from the test set, and randomly selected 12,500 tasks from them to calculate the average accuracy. The following accuracy metrics are adopted for evaluation:

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}, \quad (10)$$

where FP is false positive, TP is true positive, TN represents the true negative and FN is false negative. In the test stage, we summarize TP, TN, FN and FP based on the classification results of the predicted labels.

3) *Compared Baselines*: In the experiment, we aim to compare CGNN with other kinds of state-of-the-art methods in the few-shot classification settings. We also evaluate the proposed CGNN compared with EGNN [10], the state-of-the-art GNN-based methods for few-shot classification. These methods mainly fall into 3 categories: 1) Un-supervised methods including 3D-GAN, Latent-GAN, PointCapsNet and FoldingNet. 2) Supervised methods including PointNet++, PointCNN, PointNet and DGCNN. 3) Self-supervised method SFNet reference missing.

4) *Implementation Details*: The implementation details of the proposed CGNN and the compared baselines are given as follows. For the proposed CGNN, we adopt 5 edge convolution layers in the IFE-GNN module. Each edge convolution layer contains multiple convolution kernels and a KNN operation which selects 20 nearest neighbors for intra-object feature extraction. The max-pooling is adopted as the aggregation method. The IFE-GNN module in the CGNN contains 8 graph layers. Each graph layer contains 3 EdgeUpdate and NodeUpdate layers for relation reasoning. In section VI, we further conduct a series of experiments to show that CGNN performs better under various network depths compared with other GNN-based methods. We adopt 128-dimensional vectors to represent the edge features as well as the node features. We use the well-known Adam optimizer to train CGNN, and the learning rate of the optimizer is initialized as 10^{-3} with the weight decay of 10^{-6} .

All the compared baselines are evaluated in the few-shot learning setting. Following SFNet [34], to evaluate the performance of the unsupervised approaches (PointCapsNet and FoldingNet), the output of the last layer embedding of the trained model is evaluated based on the classification results of the classifier such as SVM. For the other supervised approaches, the weights are randomly initialized on the raw

¹<https://shapenet.cs.stanford.edu/media/>

²<https://modelnet.cs.princeton.edu/>

³<https://www.acfr.usyd.edu.au/papers/SydneyUrbanObjectsDataset.shtml>

TABLE I

FEW-SHOT CLASSIFICATION RESULTS OF 5-WAY 5-SHOT TASK ACCURACIES ON BENCHMARK DATASETS. ALL RESULTS ARE AVERAGED OVER 500 TEST EPISODES. TOP RESULTS ARE PRESENTED IN **BOLDFACE**

Methods	ModelNet40	ShapeNet	Sydney Urban
PosPool [40]	58.12	60.35	48.92
Grid-GCN [41]	46.15	49.39	43.08
PointCapsNet [42]	41.89	44.30	41.30
FoldingNet [43]	34.50	38.62	43.69
PointNet++ [6]	37.78	40.84	58.12
DGCNN [36]	32.53	35.64	47.55
SFNet [34]	65.90	69.70	68.31
EGNN [10]	72.51	77.28	65.94
CGNN (Ours)	76.85	79.63	69.49

data before training the model, and extracted feature is fed into the classifier such as SVM. For baselines that are unable to deal with point cloud data, we adopt DGCNN as the embedding method. EGNN contains the same number of graph layers as our CGNN for a fair comparison. CGNN and the baselines are implemented with Pytorch on a server with an NVIDIA Tesla P100 GPU and Intel Xeon CPU.

Note that we adopt data augmentation to bring variations to the training datasets and improve the recognition ability for all the baselines. In our experiments, we adopt schemes including translation, scaling, rotation and perturbation in the data augmentation step.

B. Performance Evaluation

1) *Few-Shot Classification Results:* We evaluate the performance of CGNN with the baselines in the few-shot classification settings on the benchmark datasets. The few-shot classification accuracy of the 5-way-5-shot tasks is presented in Table. I. We observe that the proposed CGNN achieve significant improvement over all the compared based models on the 5-way 5-shot classification task. Both the EGNN and the CGNN outperform the non-GNN-based methods, mainly because both the two methods utilize the GNNs to propagate the label information for reason reasoning, which demonstrates the effectiveness of GNNs for relation reasoning in the few-shot classification tasks. Compared with the state-of-the-art GNN-based method for few-shot learning, i.e., EGNN, CGNN further improves the prediction accuracy from 72.51 to 76.85 on ModelNet40, and from 77.28 to 79.63 on ShapeNet. We further compare the proposed CGNN with EGNN on various task settings with different numbers of ways and shots. The evaluation results are shown in Table. II. Both the CGNN and the EGNN construct the queries and supports as fully connected graphs in a similar way. Nevertheless, CGNN outperforms EGNN in different settings, which demonstrates the effectiveness of utilizing the cascade learning mechanism. An observation is that the improvements obtained by CGNN are much better in tasks with more ways and shots. The reasons for the significant improvement of CGNN over EGNN are folds: 1) Our proposed discriminative edge label can model the channel-wise (dis)similarity of the point cloud objects. 2) The proposed few-shot circle maximizes the gap between

TABLE II

FEW-SHOT CLASSIFICATION RESULTS OF DIFFERENT NUMBERS OF WAYS AND SHOTS ON THE BENCHMARK DATASETS

Methods	Number of Way	Number of Shot	Modelnet40	ShapeNet	Sydney Urban
EGNN	5	1	61.94	69.28	57.65
EGNN	5	5	72.51	77.28	65.94
EGNN	5	10	73.18	78.52	66.37
EGNN	10	5	64.16	68.72	-
CGNN	5	1	63.64	67.18	55.90
CGNN	5	5	76.85	79.63	69.49
CGNN	5	10	77.20	80.29	67.19
CGNN	10	5	69.19	71.54	-

TABLE III

EXPERIMENTAL RESULTS SUMMARY OF 5-WAY 5-SHOT TASK FOR CLASSES CLOSELY RELATED TO AUTONOMOUS DRIVING IN MODELNET40 AND SHAPENET (IN F1-SCORE)

dataset	ModelNet40		ShapeNet		
	class		class		
method	car	person	bus	car	motorcycle
CGNN	0.71	0.74	0.79	0.80	0.82
EGNN	0.69	0.71	0.77	0.79	0.73

similarity and dissimilarity between the support to support and support to query pairs. These schemes give CGNN a stronger ability to capture the features and relations in the point cloud Data.

2) *Visualization:* Fig. 8 provides an illustrative example of how our CGNN works to reason the relations of the support set and the query set by visualizing heatmaps of the pair-wise similarity of the node features of 4 layers of IRR-GNN in the inference stage. The similarities are calculated according to the methodology proposed in section III-D.5. In the beginning, the pair-wise similarities seem random and irrelevant to the class labels for both the support and query sets. As the GNN becomes deeper, the features get distinguishable and the intra-class support nodes (the diagonal elements) become similar while the inter-class nodes become dissimilar in feature space. The query set can be classified into class 1 (class of nodes 1 to 5 in the query set) with the calculated similarity of the 4th layer. As all the classes in the test stage are different from those used for training, this illustrative example demonstrates the ability of our CGNN to capture the relations and distinguish the unseen classes and predict the labels.

C. Ablation Studies

1) *Effects of the Discriminative Edge Label:* We explore the performance the variant of CGNN to evaluate the effectiveness of the proposed discriminative edge label. $CGNN_{nel}$ denotes no edge label is involved in each graph layer and only the node label is adopted in the label propagation. $CGNN_{el}$ denotes the variant in which the discriminative edge label is replaced with the edge label scheme proposed in EGNN in which a single dimension relationship value represents the overall similarity and dissimilarity and this value has the same effect on all the node features in the label propagation.

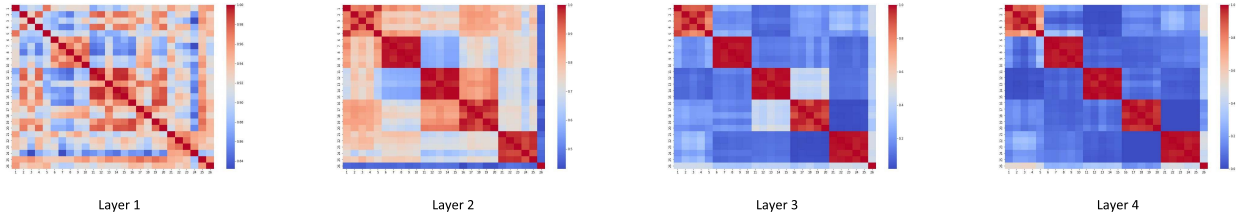


Fig. 8. Visualized heatmap of the pair-wise similarity of the node features of IRR-GNN in the inference stage of a 5-way 5-shot classification task. Figures from left to right illustrate the similarities of the 1st to the 4th layer. Nodes 1 to 25 are the support set in 5 classes while node 26 is the query set. In the 4th layer, the query node 26 has the highest similarity to nodes 1-5 and is therefore classified into class 1.

TABLE IV

EFFECTS OF THE DISCRIMINATIVE EDGE LABEL ON FEW-SHOT CLASSIFICATION. CGNN_{nel} DENOTES THE VARIANT IN WHICH NO EDGE LABEL SCHEME IS ADOPTED. CGNN_{el} DENOTES THE DISCRIMINATIVE EDGE LABEL IS REPLACED WITH THE EDGE LABEL SCHEME PROPOSED IN EGNN

Baseline	ModelNet40	ShapeNet
CGNN _{nel}	65.32	67.76
CGNN _{el}	73.19	78.58
CGNN	76.85	79.63

TABLE V

EFFECTS OF THE FEW-SHOT CIRCLE LOSS ON FEW-SHOT CLASSIFICATION

Baseline	ModelNet40	ShapeNet
CGNN _{fcl}	76.85	79.63
EGNN _{fcl}	74.8	78.4
CGNN _{bce}	68.6	76.1
EGNN _{bce}	72.51	77.28

Table IV shows the experimental results on ModelNet40 and ShapeNet datasets. The results show that CGNN outperforms CGNN_{el} and CGNN_{nel}. EGNN only uses a two-dimensional vector to represent the overall similarity and dissimilarity and this value has the same effect on all the node features in the label propagation. This formulation is inefficient for point cloud data which has richer feature relations than the 2D image data such as topological information, while our scheme can carry richer channel-wise feature relations. It demonstrates the effectiveness of our method to model the channel-wise (dis)similarity of the point cloud objects.

2) *Effects of the Few-Shot Circle Loss*: We evaluate the effectiveness of the proposed few-shot circle (FSC) loss by comparing it with the binary cross entropy (BCE) loss which is a traditional loss to supervise the training of classification task. We evaluate the two losses on CGNN and EGNN respectively. The variants used in this experiment are denoted as CGNN_{fsc}, CGNN_{bce}, EGNN_{bce}, and EGNN_{fsc}. The experiment results are summarized in Table V. We observed that the proposed few-shot circle outperforms the binary cross entropy loss in both the baselines, mainly because our proposed few-shot circle while our few-shot circle loss maximizes the gap between similarity and dissimilarity between the support to support and support to query pairs. The results have also demonstrated the proposed loss has great potential to be a general scheme in the few-shot learning tasks.

TABLE VI

INFLUENCE OF THE NETWORK DEPTH ON FEW-SHOT CLASSIFICATION ON MODELNET40

Network Depth	2	4	6	8	10
l_1	74.97	75.60	75.62	75.20	75.01
l_2	73.24	74.62	75.87	76.85	76.20

3) *Evaluation on Network Depth*: Network depth is a critical hyperparameter that directly determines the quality of the feature learning. Our proposed CGNN is a cascade learning framework, it is necessary to separately determine the network depths of the IFE-GNN l_1 and the IRR-GNN l_2 . Here we first fix the l_2 as 8 to obtain the best value of l_1 , and then fix l_1 to obtain l_2 . Table VI presents the performance for different numbers of network depths. From the results, we can observe that both the performance increases with deeper GNN layers before the depth of 4 for IFE-GNN and 8 for IRR-GNN. However, we observe a degradation in performance as the networks become even deeper than these thresholds. This degradation is possibly caused by the over-smoothing problem in GNNs, which tends to make all the nodes similar in the feature space in deep layers and losses the discriminative information.

D. Discussion for Autonomous Driving Application

Identifying rare and emerging classes is critical for safety in autonomous driving [44]. Here we discuss the opportunities for the proposed method in autonomous driving. The classes in Sydney Urban datasets are all collected in real urban scenarios. Moreover, both the ModelNet40 and the ShapeNet datasets contain rich point cloud objects in the urban road environments, such as cars, buses, pedestrians, motors and so on. Table III summarizes the evaluation results over classes closely related to autonomous driving in ModelNet40 and ShapeNet in f1-score, including “bus”, “car”, “motorcycle”, and “person”. Note that all these classes are selected for testing. We outperform EGNN for all these classes. The results have demonstrated the effectiveness of our CGNN to recognize the classes related to autonomous driving in both the real LiDAR and the CAD point cloud datasets in the few-shot setting. Another issue is that the proposed classification method requires the 3D bounding boxes in advance. A possible solution in practice is to leverage the open-set 3D object detection methods [45] which predict an “unknown” label and

a 3D bounding box for the object whose class is not in the training set, and use the proposed CGNN after the open-set 3D detection method to further classify the “unknown” class into specific classes.

V. CONCLUSION

This work proposes a novel cascade graph neural networks (GNN) for few-shot learning (FSL) on point clouds, termed as CGNN. There are two cascade parts in CGNN for both the intra-object topological feature extraction and the inter-object relation reasoning of the point cloud objects with a novel discriminative edge label scheme that models the channel-wise similarity of the point cloud objects. We also propose a novel few-shot circle loss to maximize the gap between similarity and dissimilarity between the support to support and support to query pairs, which have a great potential to be a general scheme in the few-shot learning tasks. Extensive experiments on benchmark point cloud datasets have demonstrated that CGNN outperforms the state-of-the-art FSL classification methods on point clouds significantly. In the future, we will study to improve the performance of FSL on point cloud data with incomplete shapes.

REFERENCES

- [1] Y. Guo, H. Wang, Q. Hu, H. Liu, L. Liu, and M. Bennamoun, “Deep learning for 3D point clouds: A survey,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 12, pp. 4338–4364, Dec. 2021.
- [2] R. B. Rusu, Z. C. Marton, N. Blodow, M. Dolha, and M. Beetz, “Towards 3D point cloud based object maps for household environments,” *Robot. Auton. Syst.*, vol. 56, no. 11, pp. 927–941, Nov. 2008.
- [3] C. R. Qi, W. Liu, C. Wu, H. Su, and L. J. Guibas, “Frustum PointNets for 3D object detection from RGB-D data,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 918–927.
- [4] R. M. Rustamov, “Laplace–Beltrami eigenfunctions for deformation invariant shape representation,” in *Proc. 5th Eurograph. Symp. Geometry Process.* Eurographics Association, 2007, pp. 225–233.
- [5] R. Q. Charles, H. Su, M. Kaichun, and L. J. Guibas, “PointNet: Deep learning on point sets for 3D classification and segmentation,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 652–660.
- [6] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “PointNet++: Deep hierarchical feature learning on point sets in a metric space,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5099–5108.
- [7] Y. Li, R. Bu, M. Sun, W. Wu, X. Di, and B. Chen, “PointCNN: Convolution on X-transformed points,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 820–830.
- [8] Y. Zhang, Q. Hu, G. Xu, Y. Ma, J. Wan, and Y. Guo, “Not all points are equal: Learning highly efficient point-based detectors for 3D LiDAR point clouds,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 18953–18962.
- [9] X. Chen, H. Ma, J. Wan, B. Li, and T. Xia, “Multi-view 3D object detection network for autonomous driving,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1907–1915.
- [10] J. Kim, T. Kim, S. Kim, and C. D. Yoo, “Edge-labeling graph neural network for few-shot learning,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 11–20.
- [11] A. Li, T. Luo, Z. Lu, T. Xiang, and L. Wang, “Large-scale few-shot learning: Knowledge transfer with class hierarchy,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 7212–7220.
- [12] J. Snell, K. Swersky, and R. Zemel, “Prototypical networks for few-shot learning,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 4077–4087.
- [13] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales, “Learning to compare: Relation network for few-shot learning,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1199–1208.
- [14] J. B. V. Garcia, “Few-shot learning with graph neural networks,” in *Proc. Int. Conf. Learn. Represent.*, 2018.
- [15] O. Vinyals et al., “Matching networks for one shot learning,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 3630–3638.
- [16] C. Zhang, C. Li, and J. Cheng, “Few-shot visual classification using image pairs with binary transformation,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 30, no. 9, pp. 2867–2871, Sep. 2020.
- [17] Y. Wang, Q. Yao, J. Kwok, and L. M. Ni, “Generalizing from a few examples: A survey on few-shot learning,” 2019, *arXiv:1904.05046*.
- [18] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [19] A. Cheraghian, S. Rahman, T. F. Chowdhury, D. Campbell, and L. Petersson, “Zero-shot learning on 3D point cloud objects and beyond,” *Int. J. Comput. Vis.*, vol. 130, no. 10, pp. 2364–2384, Oct. 2022.
- [20] Y. Gao, X. Han, X. Wang, W. Huang, and M. Scott, “Channel interaction networks for fine-grained image categorization,” in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 7, pp. 10818–10825.
- [21] J. Yosinski, J. Clune, A. Nguyen, T. Fuchs, and H. Lipson, “Understanding neural networks through deep visualization,” 2015, *arXiv:1506.06579*.
- [22] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–14.
- [23] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–12.
- [24] H. Gao and S. Ji, “Graph U-Nets,” in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2083–2092.
- [25] W. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1024–1034.
- [26] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1126–1135.
- [27] S. Ravi and H. Larochelle, “Optimization as a model for few-shot learning,” in *Proc. Int. Conf. Learn. Represent.*, 2017.
- [28] H. Li, D. Eigen, S. Dodge, M. Zeiler, and X. Wang, “Finding task-relevant features for few-shot learning by category traversal,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 1–10.
- [29] C. Chen, K. Li, W. Wei, J. T. Zhou, and Z. Zeng, “Hierarchical graph neural networks for few-shot learning,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 1, pp. 240–252, Jan. 2022.
- [30] C. Chen, K. Li, S. G. Teo, X. Zou, K. Li, and Z. Zeng, “Citywide traffic flow prediction based on multiple gated spatio-temporal convolutional neural networks,” *ACM Trans. Knowl. Discovery Data*, vol. 14, no. 4, pp. 1–23, Aug. 2020.
- [31] W. Zhang, H. Quan, and D. Srinivasan, “An improved quantile regression neural network for probabilistic load forecasting,” *IEEE Trans. Smart Grid*, vol. 10, no. 4, pp. 4425–4434, Jul. 2019.
- [32] W. Zhang, S. Liu, O. Gandhi, C. D. Rodríguez-Gallegos, H. Quan, and D. Srinivasan, “Deep-learning-based probabilistic estimation of solar PV soiling loss,” *IEEE Trans. Sustain. Energy*, vol. 12, no. 4, pp. 2436–2444, Oct. 2021.
- [33] T. Weng, X. Zhou, K. Li, P. Peng, and K. Li, “Efficient distributed approaches to core maintenance on large dynamic graphs,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 1, pp. 129–143, Jan. 2022.
- [34] C. Sharma and M. Kaul, “Self-supervised few-shot learning on point clouds,” 2020, *arXiv:2009.14168*.
- [35] C. Liu, K. Li, K. Li, and R. Buyya, “A new service mechanism for profit optimizations of a cloud provider and its users,” *IEEE Trans. Cloud Comput.*, vol. 9, no. 1, pp. 14–26, Jan. 2021.
- [36] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, “Dynamic graph CNN for learning on point clouds,” *ACM Trans. Graph.*, vol. 38, no. 5, pp. 1–12, 2019.
- [37] Y. Sun et al., “Circle loss: A unified perspective of pair similarity optimization,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 6398–6407.
- [38] Z. Wu et al., “3D ShapeNets: A deep representation for volumetric shapes,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1912–1920.
- [39] A. X. Chang et al., “ShapeNet: An information-rich 3D model repository,” 2015, *arXiv:1512.03012*.
- [40] Z. Liu, H. Hu, Y. Cao, Z. Zhang, and X. Tong, “A closer look at local aggregation operators in point cloud analysis,” in *Proc. Eur. Conf. Comput. Vis.* Springer, 2020, pp. 326–342.

- [41] Q. Xu, X. Sun, C.-Y. Wu, P. Wang, and U. Neumann, "Grid-GCN for fast and scalable point cloud learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 5661–5670.
- [42] Y. Zhao, T. Birdal, H. Deng, and F. Tombari, "3D point capsule networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 1009–1018.
- [43] Y. Yang, C. Feng, Y. Shen, and D. Tian, "FoldingNet: Point cloud auto-encoder via deep grid deformation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 206–215.
- [44] K. Wong, S. Wang, M. Ren, M. Liang, and R. Urtasun, "Identifying unknown instances for autonomous driving," in *Proc. Conf. Robot Learn.*, 2020, pp. 384–393.
- [45] J. Cen, P. Yun, J. Cai, M. Y. Wang, and M. Liu, "Open-set 3D object detection," in *Proc. Int. Conf. 3D Vis. (3DV)*, Dec. 2021, pp. 869–878.



Zhongyao Cheng is a Research Engineer with I2R, A*star, Singapore. He has provided technique support for both academic articles and industrial projects and has published several papers regarding to deep learning and computer vision. His research interests include computer vision and deep learning.



Yangfan Li received the bachelor's degree in engineering from the School of Automation, Huazhong University of Science and Technology, in 2015, and the Ph.D. degree in computer science from Hunan University, China, in 2022. He is currently a Lecturer with the School of Computer Science and Engineering, Central South University, China. His research interests include computer architecture, efficient machine learning, and deep learning.



Hui Li Tan received the B.Sc. degree in applied mathematics and the Ph.D. degree in electrical and computer engineering from the National University of Singapore (NUS), Singapore, in 2007 and 2017, respectively. Since 2007, she has been with the Institute for Infocomm Research, Singapore. Her current research interests include computer vision, multi-modal deep learning, incremental, and federated learning.



Cen Chen received the Ph.D. degree in computer science from Hunan University, China. Currently, he is a Scientist II with the Institute for Infocomm Research (I2R), Agency for Science, Technology and Research (A*STAR), Singapore. His research interests include parallel and distributed computing, machine learning, and deep learning.



Weiquan Yan received the bachelor's degree in automation from Shenzhen University, Shenzhen, China, in 2015, and the master's degree from the National University of Singapore, Singapore, in 2021. He joined as an Algorithm Engineer with the Peng Cheng Laboratory in 2021. His research interests include EEG data analysis for brain-computer interaction applications in intelligent systems and low-level neuromorphic vision task.



Wenjie Zhang received the B.Eng. degree from the School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan, China, in 2015, and the Ph.D. degree in electrical and computer engineering from the National University of Singapore (NUS), Singapore, in 2020. In 2019, he was a Visiting Scholar with Stanford University, Stanford, CA, USA. He is currently involved in numerous AI-oriented projects. His current research interests include uncertainty quantification and deep learning in smart cities and smart grids.